◆ TICKET PULSE

# Comprehensive QA Test Plan

Build v3.0.71-preview + hardening pass · Test cycle 2026-07-20

Prepared for the QA team · generated 2026-07-18

## Ticket Pulse — Comprehensive QA Test Plan (2026-07-20)

**Build under test:** v3.0.71-preview (prod) · **Prepared for:** QA team · **From:** Ticket Pulse dev

This single plan consolidates everything shipped since the last round so it can be tested end-to-end in one pass:

1. **Custom Agent Alerts** (per-agent alert subscriptions)
2. **Email delivery health** (visibility + the SendGrid fix)
3. **Tasks, Activity & Notifications** (the July 17 batch)
4. **Public API** (integration keys, OAuth2, rate limits, webhooks) — new

Use a **QA TEST** prefix on any tickets you create and clean them up as usual. Report anything that deviates from the ✅ expected result, with the ticket ref and (for API items) the `X-Request-Id` from the response.

> This revision folds in a **hardening pass** over the whole batch (security, data integrity, and reliability). Steps updated by that pass are flagged inline — notably test-mode keys are now read-only (§4.8), repeat escalations re-alert (§1.4), idempotency handles concurrent retries (§4.5), and webhooks support secret rotation + private-target guards (§4.10).

---

## 1. Custom Agent Alerts

Agents self-serve alert subscriptions in their portal: **My Competencies → Notifications → My alerts** (note: this used to be a separate "My Alerts" tab — see §3.4 for that change).

### 1.1 Create a subscription

1. Log in as an agent → **Notifications → My alerts → Add an alert**.
2. Pick a Category you can generate test tickets for (e.g. Licensing). Leave Tag/Priority as "Any". Leave **arrives (new)** and **Email** checked. Create.
3. ✅ It appears listing the scope, triggers, and channel icons.

### 1.2 New-ticket alert

1. Create a **QA TEST** ticket in that category.
2. ✅ Within ~30–60s you receive **one email**: "1 new ticket in <category>".

### 1.3 Alert-storm protection

1. Quickly create **several** QA TEST tickets in the same category.
2. ✅ You get **one grouped email** ("N new tickets in <category>"), not one per ticket.

### 1.4 Escalation & re-categorization

1. Edit the subscription: check **is escalated** and **is re-categorized into scope**.
2. Raise a matching ticket's priority to High/Urgent → ✅ escalation alert.
3. Move a ticket's category into your watched category → ✅ re-categorized alert. This fires whether the category change is made **by AI or manually** (a coordinator editing the category counts).
4. **Repeat escalation:** raise the *same* ticket again (e.g. High → Urgent after the first alert was delivered). ✅ you get a **second** alert for the new escalation — a further raise on the same ticket is no longer permanently suppressed.

### 1.5 Priority filter, channels, quiet hours

1. Set **Priority = Urgent only**: a Medium ticket in scope → ✅ no alert; an Urgent → ✅ alert.
2. (Needs a verified phone in the Notifications tab) enable **SMS/WhatsApp/Phone** → ✅ delivered in addition to email. Unverified channels are greyed out.
3. Enable **Quiet hours** covering "now": a non-urgent ticket → ✅ no alert during the window, arrives after it ends; an Urgent (if "let Urgent through" is on) → ✅ immediate.

### 1.6 Pause / edit / delete

- ✅ Pausing stops alerts until resumed; edit changes scope/triggers/channels; delete removes it.

> **Note:** there's a deliberate grouping delay (up to ~60–90s) so bursts collapse; a single ticket alerts within about a minute, not instantly. Within a single grouping window the same ticket won't alert twice for the same trigger — but a **new** event on that ticket later (e.g. a further escalation, or a re-categorization after the first alert was sent) will alert again. Alerts that fail to deliver (e.g. email outage) are retried rather than silently consumed.

---

## 2. Email delivery health

Every outbound email is now checked and surfaced, so a provider outage can't fail silently.

### 2.1 The health card

1. As an admin → **Settings** → **Notifications**.
2. ✅ An **"Email delivery health"** card shows a status badge (Healthy / Degraded / Delivery failing), last successful send, last-24h sent/failed counts, and — if failing — a plain-English hint.

### 2.2 Prove a successful send registers

1. In **Settings** → **Notifications** → **SendGrid**, send a test email to yourself.
2. ✅ It arrives, and the health card shows a recent success / bumped 24h count.

### 2.3 Re-test the flows that were failing

The earlier "no email received" reports (approvals, task-assignment, merge "email each requester", My Alerts, urgent tickets) were a single SendGrid block that's now fixed. Please re-run each and confirm the email now arrives:

1. An approval with **email approver** checked. ✅ approver gets the email.
2. Assign a **task** to yourself with notify on (§3.1). ✅ assignment email arrives.
3. A **merge** with "email each requester". ✅ requester emails arrive.

4. A **My Alerts** trigger (§1.2). ✅ alert email arrives.
5. An **urgent** ticket in a watched category. ✅ alert email arrives.

> If any email doesn't arrive, open the health card first — its status + hint point straight at the cause. (Admins also get an app-wide banner if delivery is failing.)

### 2.4 SMS/voice health (if Twilio is configured)

1. With an alert subscription that has **SMS** (or WhatsApp/Phone) enabled and a verified phone, trigger an alert.
2. ✅ SMS send attempts are now tracked in delivery health too (channel `sms`), so a silent Twilio drop is visible rather than swallowed — the same protection email got.

---

## 3. Tasks, Activity & Notifications (July 17 batch)

### 3.1 Task form + status + editing

1. Open any ticket → **Tasks** tab.
2. ✅ The add-task row shows labels: **Task Description**, **Assign To**, **Due Date**.
3. Add a task, assign it to yourself, notify on. ✅ It appears; you get the email (§2.3).
4. On the task, use the **status dropdown** (Open / In progress / Done). ✅ status updates.
5. Click the **pencil** to edit the task's wording inline and save. ✅ text updates.

### 3.2 Status changes log to Activity

1. Change a task's status.
2. Open the ticket's **Activity** tab.
3. ✅ You see the change logged (e.g. "Task status changed — Open → Done · Task: ...").

### 3.3 Task status syncs with FreshService (both ways)

1. On a FreshService-linked ticket, mark a task **Done** in Ticket Pulse. ✅ FreshService shows it Done.
2. Mark a task **Done in FreshService**, then reopen the ticket's **Tasks** tab in Ticket Pulse. ✅ it shows Done (statuses reconcile when the tab loads).

### 3.4 "Activity" naming + merged Notifications tab

1. On a ticket, the detail tab is now called **Activity** (was "History"), matching the preview drawer. ✅ consistent name.
2. In the agent portal there is now a single **Notifications** tab with two sub-sections — **Notification preferences** and **My alerts** — instead of two separate tabs. ✅ both live under one tab.

---

## 4. Public API (new — integration surface)

The `/api/v1` API is now a FreshService-replacement surface. **Docs:** `https://<app>/api/v1/docs` · **Spec:** `/api/v1/openapi.json`. All API access is **scoped to one workspace** — a credential can never read or write another workspace's data.

**Setup (admin):** Settings → **API Keys**. You'll see three sections: **Integration API keys**, **OAuth clients**, and **Outbound webhooks**.

### 4.1 API keys — create, scopes, test/live

1. Create an API key named "QA test" with **Live** mode, no expiry, and scopes `tickets:read`, `tickets:write`, `conversations:write`, `tasks:write`. Copy the key (shown once — starts `tp_live_`).
2. Confirm identity:

`` `curl https://<app>/api/v1/me -H "Authorization: Bearer tp_live_…"` ✅ returns your key name, **workspace**, scopes, and authType: api_key`.

1. ✅ Every response has an `X-Request-Id` and `X-RateLimit-Limit/Remaining/Reset` header.

### 4.2 Create & read tickets

1. `POST /api/v1/tickets` with `{ "subject":"QA TEST via API", "requesterEmail":"you@…", "priority":2 }`. ✅ 201 with the created ticket (`ref` like `TP-####`).
2. `GET /api/v1/tickets/<id>` ✅ returns the ticket + its public conversation.
3. `GET /api/v1/tickets?pageSize=5` ✅ returns 5 with a `pagination` block; try `?cursor=<next_cursor>` from the response ✅ returns the next page.

### 4.3 Update, reply, tasks

1. `PATCH /api/v1/tickets/<id>` with `{ "status":"Pending", "priority":3 }`. ✅ updated.
2. `POST /api/v1/tickets/<id>/replies` `{ "body":"Hello from the API" }`. ✅ 201; the requester is emailed (verify via §2).
3. `POST /api/v1/tickets/<id>/tasks` `{ "title":"QA API task" }` ✅ 201; `GET …/tasks` lists it.

### 4.4 Scopes are enforced (deny-by-default)

1. With the QA key (which lacks `tags:write`), call `PUT /api/v1/tickets/<id>/tags`. ✅ **403 problem+json** with `code: insufficient_scope`.
2. Call any endpoint with **no** Authorization header. ✅ **401 problem+json** (`code: api_key_required`), `Content-Type: application/problem+json`.

### 4.5 Idempotency (retry safety)

1. `POST /api/v1/tickets` **twice** with the same header `Idempotency-Key: qa-123` and identical body. ✅ **one** ticket is created; the second response is the same as the first (header `Idempotent-Replayed: true`), no duplicate.
2. Reuse `Idempotency-Key: qa-123` with a **different** body. ✅ **422** (`idempotency_key_reused`).
3. **Concurrent duplicates:** fire two identical `POST /tickets` with the same fresh `Idempotency-Key` at the same moment (e.g. two parallel curls). ✅ exactly **one** ticket is created; the loser returns **409** (`idempotency_in_flight`) rather than a second ticket. (Idempotency also works for OAuth-token callers now, not just API keys.)

### 4.6 Rate limiting

1. Rapidly send >120 requests in a minute with one key (a quick loop against `/me`). ✅ you start getting **429 problem+json** with a `Retry-After` header and `X-RateLimit-Remaining: 0`.

### 4.7 OAuth2 client-credentials

1. Settings → API Keys → **OAuth clients** → **New OAuth client** "QA daemon", scopes `tickets:read`, `tickets:write`. Copy the **client_id** (`tpc_…`) and **client_secret** (`tps_…`, shown once).
2. Exchange for a token:

`curl -X POST https://<app>/api/v1/oauth/token \ -d grant_type=client_credentials -d client_id=tpc_… -d client_secret=tps_…` ✅ `returns` { access_token, token_type:"Bearer", expires_in:3600, scope }.

1. Use the token: `GET /api/v1/me -H "Authorization: Bearer <access_token>"`. ✅ works, `authType: oauth`, correct workspace + scopes.
2. Wrong secret at the token endpoint → ✅ **401** `{ "error":"invalid_client" }`.
3. **Disable** the OAuth client in Settings, then reuse the token. ✅ **401** (revocation is immediate — the token is re-checked against the live client).

### 4.8 Test-mode key (read-only)

1. Create a key in **Test** mode (`tp_test_…`). ✅ clearly labeled **Test** in the list.
2. Use it for a **read**, e.g. `GET /api/v1/tickets`. ✅ works.
3. Use it for a **write**, e.g. `POST /api/v1/tickets`. ✅ **403 problem+json** (`code: test_mode_read_only`) — test keys are read-only, so you can build against the API without any risk of touching live tickets or emailing real requesters. Switch to a `tp_live_…` key to make changes.

### 4.9 Key rotation & expiry

1. **Rotate** a key. ✅ a new secret is shown once; the old secret stops working; the key keeps its enabled/disabled state.
2. **Revoke** a key (Disable), then try to **Rotate** it. ✅ rotation is **refused** with a clear message — rotating no longer silently re-arms a revoked key (create a new one).
3. Create a key that **expires in 30 days**. ✅ the list shows the expiry date.
4. Deleting or rotating a key/client now asks for **confirmation** first (guarding against an accidental one-click on the trash/rotate icon).

### 4.10 Outbound webhooks (durable + signed)

1. **Outbound webhooks → New webhook**. Point the URL at a request-capture tool (e.g. webhook.site), check `ticket.created`, save, copy the signing secret (`whsec_…`).
2. **Test** the webhook. ✅ a signed `ping` arrives with headers `webhook-id`, `webhook-timestamp`, `webhook-signature` (`v1,<hmac>`) — and a legacy `X-TicketPulse-Signature` alongside during migration.
3. Create a **QA TEST** ticket. ✅ a `ticket.created` delivery arrives with the full ticket embedded (no need to re-fetch). The `webhook-signature` verifies with a standard Standard-Webhooks library (the secret is now standard base64, so strict verifiers — e.g. Python — validate it correctly).
4. Point a webhook at a URL that returns 500, trigger an event, then open the webhook's **Log**. ✅ you see the failed attempts with backoff; click **Redeliver** to retry. ✅ 20 consecutive dead deliveries auto-disable the hook (with the reason shown).
5. **Secret rotation:** rotate the webhook's signing secret. ✅ a new secret is shown once; for the next **24h** deliveries carry **both** signatures (space-separated in `webhook-signature`) so you can roll consumers over without dropping events.
6. **Private-target guard:** try to save a webhook pointing at an internal address (e.g. `http://169.254.169.254/…`, `http://127.0.0.1`, or the decimal form `http://2130706433/`). ✅ rejected as a private/internal address; deliveries also refuse to follow a redirect that lands on one.

---

## Notes for QA

- API base host is the app URL (`/api/v1/*`). The interactive docs at `/api/v1/docs`

list every endpoint, its scope, and copy-paste curl for both auth methods.

- Please report: any endpoint that returns another workspace's data; a scope that

isn't enforced; a duplicate created despite an Idempotency-Key; a webhook signature that doesn't verify; or an email that still doesn't arrive (with the health-card status).